① Explain the RTL concept in digital system design.

Ans : RTL stands for Register Transfer Level.

It is a level of abstraction in digital system design where the behavior of a system is described in terms of transfers of data between registers and operations performed on the data.

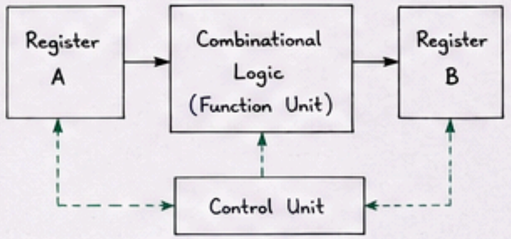
1. Meaning of RTL

- RTL describes how data moves between registers and how it is processed by the combinational logic.
- It is more abstract than logic (gate) level but more detailed than behavioral level.
- RTL models the system in terms of registers, combinational logic and control signals.

2. Basic Elements in RTL

(i) Registers : Used to store binary data.
(ii) Combinational Logic : Performs arithmetic, logic, shifting, etc.
(iii) Data Transfer : Movement of data from one register to another OR between a register and logic unit.
(iv) Control Signals : Control the transfer and operation (e.g., Load, Clear, Enable, Select, etc.)

3. General RTL Model



(RTL Model of a Digital System)

4. RTL Representation

RTL is represented using :

- Register Transfer Language
- RTL Diagrams
- State tables/State diagrams

5. Register Transfer Notation

- $R_1 \leftarrow R2$ (Transfer content of R2 to R1)
- $R_1 \leftarrow R2 + R3$ (Addition result stored in R1)
- $R_1 \leftarrow R2 - R3$ (Subtraction result stored in R1)
- $R_1 \leftarrow R2$ AND R3 (Logic AND)
- $R_1 \leftarrow R2$ OR R3 (Logic OR)
- $R_1 \leftarrow R2$ (Shift Left)
- $R_1 \leftarrow$ shr R2 (Shift Right)

6. Example of RTL Description

Consider : $R_1 \leftarrow R2 + R3$

Steps :
(1) Read contents of R2 and R3.
(2) Send them to the adder.
(3) Sum is stored in R_1 on control signal.

RTL Notation : $R_1 \leftarrow R2 + R3$
This indicates transfer of the result of R2 + R3 to R_1.

7. Advantages of RTL

- Easy to design and modify.
- Reduces design complexity.
- Acts as a bridge between behavioral description and logic design.
- Facilitates design verification.
- Suitable for medium and large scale digital systems.

8. Applications of RTL

- Used in data path design of CPUs, ALUs, Controllers.
- Used in design of calculators, DSP systems, etc.
- Forms the basis for logic synthesis in CAD tools.

9. Difference between RTL and other levels

Design Level	Description
Behavioral Level	Describes what the system does.
RTL	Describes how data is transferred between registers and operations.
Logic Level	Describes implementation using logic gates.
Physical Level	Describes actual layout and wiring.

10. Summary

RTL is an abstraction level where a digital system is described using registers, logic operations and data transfers between them. It is the most widely used representation for the design and analysis of digital systems.

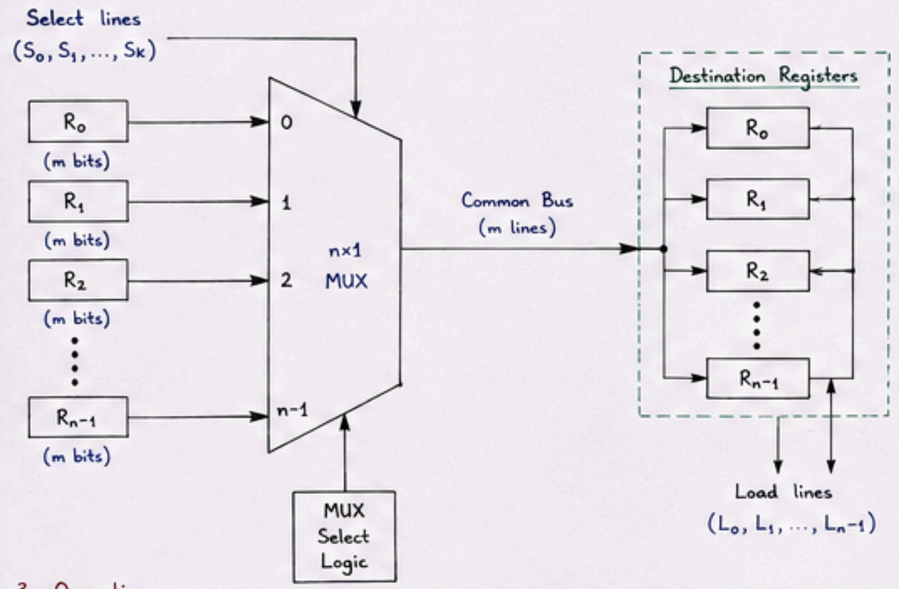
② Describe the common bus system using multiplexers with help of neat diagram.

Ans : A common bus system allows multiple registers to share a single set of data lines (bus) for transferring data. Multiplexers are used to connect one of the registers to the bus at a time.

1. Concept

- In a system with n registers, instead of using n×m lines (where m = word length) to connect all registers to each other, a common bus of m lines is used.
- Multiplexer selects one register and connects its contents to the bus.
- The selected data can then be loaded into any destination register.

2. Block Diagram using Multiplexers



Where,
n = Number of registers
m = Number of bits in each register
S = Select lines for MUX
L = Load (enable) lines
Bus = Common data bus

3. Operation

(i) The select lines (S) are set to select one of the registers.
(iii) The content of the selected register is connected to the bus through the multiplexer.
(iii) The data on the bus can be loaded into any destination register by enabling its load line.

4. Example (4 registers of 4 bits)

For 4 registers R_0, R_1, R2, R3 each of 4 bits.

- Use 4×1 MUX with select lines S_1, S_0.
- The output of MUX is connected to 4-bit common bus.
- Any register can drive the bus; any register can be loaded.

5. Select Table for 4×1 MUX (Example)

S_1	S_0	Selected Register on Bus
0	0	R_0
0	1	R_1
1	0	R_2
1	1	R_3

6. Advantages

- Reduces hardware (wires).
- Simplifies interconnections.
- Scalable for large number of registers.
- Economical in terms of area and cost.

7. Disadvantages

- Only one register can send data on bus at a time.
- Speed limited by MUX delay.

8. Applications

- CPU register file.
- Data transfer in microprocessors.
- Used in ALU, data paths, DSP systems.

Summary : A common bus system uses multiplexers to connect one of the registers to the common bus. It reduces the number of lines and simplifies design.

③ Describe the common bus system using three state buffers with help of neat diagram.

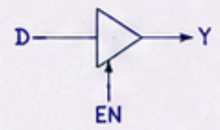
Ans :

1. Concept :

- In a common bus system, all registers are connected to the bus through three state buffers.
- A three state buffer has three possible states : 1 → Logic 1, 0 → Logic 0 and Z → High impedance.
- Only the selected register's buffer is enabled to put its content on the bus. All other buffers are in High impedance (Z) state.
- Hence, only one source drives the bus at a time, avoiding contention.

Three State Buffer

Symbol :

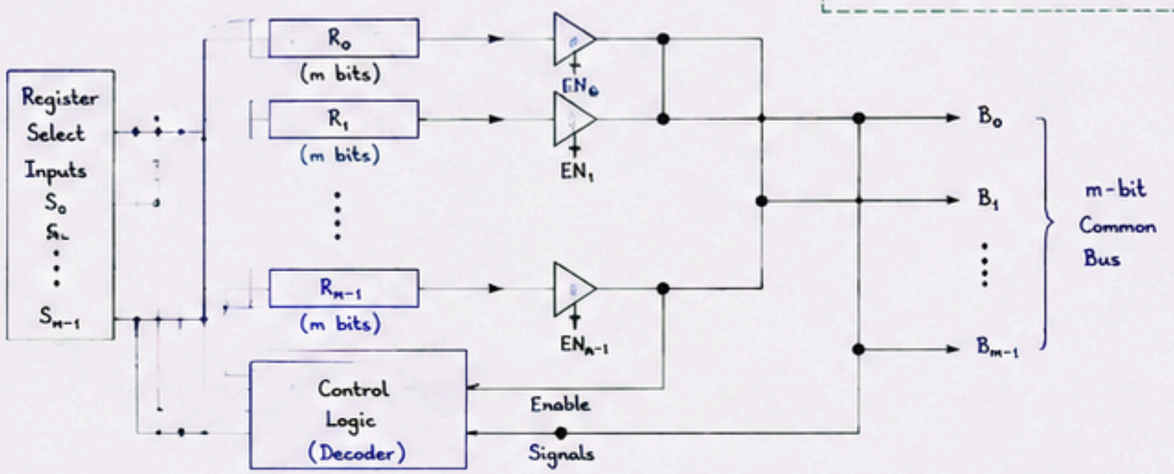


Truth Table :

EN	D	Y
0	X	Z
1	0	0
1	1	1

Z = High Impedance

2. Block Diagram (n registers on m-bit bus using three state buffers)



3. Operation :

(i) The select lines ($S_0 \ldots, S_{n-1}$) choose one register through the decoder.

(iii) The decoder enables only one buffer (EN = 1) corresponding to the selected register.

(iii) That register places its content on the bus.

(iv) All other buffers are disabled (EN = 0) and are in Z (High Impedance), so they do not affect the bus.

(v) The bus content can be loaded into any destination register by asserting its Load (enable) signal.

Key Point : Only one register drives the bus at any time.

4. Example: (4 registers of 4 bits connected to common bus

S_1	S_0	Enabled Buffer	Register on Bus
0	0	EN_0	R_0
0	1	EN_1	R_1
1	0	EN_2	R_2
1	1	EN_3	R_3

5. Advantages

- Reduces the number of bus lines to m.
- Only one register drives the bus → avoids contention.
- Easy expansion of system.
- Less hardware compared to multiplexer based system.

6. Disadvantages

- Requires 3-state buffers and decoder → more hardware.
- Slight delay due to buffer and decoder.
- Bus capacitance increases with number of registers.

7. Applications

- Used in CPU register file.
- Used in microprocessors and ALU design.
- Data transfer among registers.

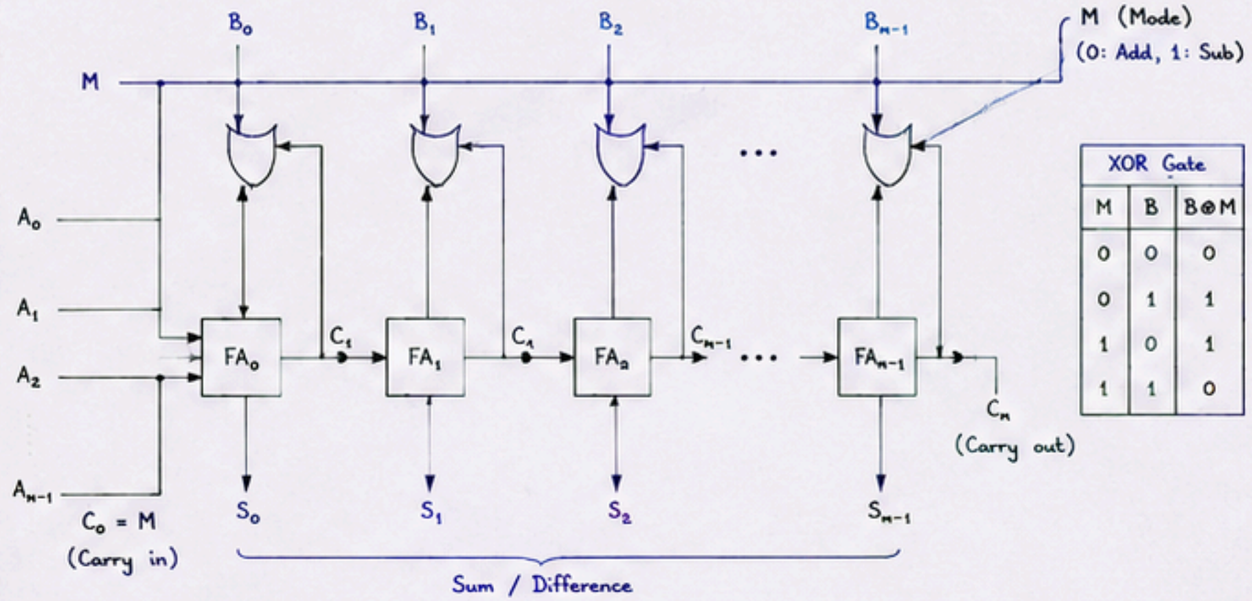
④ Draw the binary adder-subtractor circuit and explain its operation.

Ans :

1. Concept :

- The adder-subtractor performs both addition (A + B) and subtraction (A − B) using the same hardware.
- It uses n full adders connected in cascade.
- A mode control input M determines the operation.
  M = 0 → Addition M = 1 → Subtraction

2. Circuit Diagram (n-bit Adder-Subtractor)



3. Operation

(ii) Addition (M = 0)

- XOR gates pass B unchanged ($B \oplus 0 = B$).
- $C_0 = 0$
- Circuit performs A + B.
- C_n is the carry output.

(ii) Subtraction (M = 1)

- XOR gates complement B ($B \oplus 1 = \bar{B}$).
- $C_0 = 1$
- Circuit performs $A + \bar{B} + 1 = A - B$.
- $C_n = 1$ → No borrow
  $C_n = 0$ → Borrow

Note : Subtraction is done by adding the 2's complement of B to A.

4. Truth Table (Function)

M (Mode)	Operation	B Input to Adder	C_0	Function Performed	Interpretation of C_n
0	Addition	B	0	A + B	Carry out
1	Subtraction	$\bar{B}$ (complement of B)	1	$A + \bar{B} + 1 = A - B$	1 → No borrow 0 → Borrow

5. Example : (4-bit)

Let A = 0110 , B = 0011

(i) Addition (M = 0)

A + B = 0110 + 0011 = 1001 ($C_n = 0$)

(iii) Subtraction (M = 1)

A − B = 0110 − 0011 = 0011 ($C_n = 1$, No borrow)

(Check : 0110 + 1100 + 1 = 1 0011)

6. Advantages

- One circuit performs both addition and subtraction.
- Saves hardware.
- Simple and efficient.

7. Applications

- Used in ALU of CPU.
- Used in calculators, microprocessors.
- Used in DSP systems.

Summary : The adder-subtractor uses XOR gates and an initial carry to perform addition and subtraction using the same adder hardware.

⑤ Describe logic microoperations. Explain any four logic microoperations with examples.

Ans :

1. Logic Microoperations :

A microoperation is a basic operation performed on data in registers.
Logic microoperations perform Boolean (logic) functions on binary data.
They are performed by the ALU using logic gates.
General form : $R \leftarrow R \ op \ S$ (Result stored in register R)
where R and S are registers and op is a logic operation.
Common logic microoperations are : AND, OR, XOR, Complement, Clear, Set, etc.

> **Key Points :**
> - Logic microoperations perform Boolean functions.
> - They are basic operations for data manipulation.
> - Result depends on the selected operation.

2. Four Logic Microoperations (with examples)

(ii) AND Microoperation

- Performs bit-by-bit logical AND of two registers.
- Symbol : $R \leftarrow R \wedge S$
- Example : Let R = 1011, S = 1101

R	1	0	1	1	
S	1	1	0	1	
R ∧ S	1	0	0	1	→ Result in R

(ii) OR Microoperation

- Performs bit-by-bit logical OR.
- Symbol : $R \leftarrow R \vee S$
- Example : Let R = 1011, S = 1101

R	1	0	1	1	
S	1	1	0	1	
R ∨ S	1	1	1	1	→ Result in R

(iii) XOR Microoperation

- Performs bit-by-bit exclusive-OR.
- XOR gives 1 when the bits are different.
- Symbol : $R \leftarrow R \oplus S$
- Example : Let R = 1011, S = 1101

R	1	0	1	1	
S	1	1	0	1	
R ⊕ S	0	1	1	0	→ Result in R

(iv) Complement Microoperation

- Inverts (complements) all bits of a register.
- Symbol : $R \leftarrow \overline{R}$ (R-bar)
- Example : Let R = 1011

R	1	0	1	1	
$\overline{R}$	0	1	0	0	→ Result in R

3. Other Logic Microoperations (Briefly)

Microoperation	Symbol	Description	Example (4-bit)
Clear	$R \leftarrow 0$	Sets all bits of R to 0	R = 1011 → 0000
Set	$R \leftarrow 1$	Sets all bits of R to 1	R = 1011 → 1111
Copy	$R \leftarrow S$	Copies contents of S into R	S = 1101 → R = 1101
NAND	$R \leftarrow R \ NAND \ S$	$R = (R \ AND \ \overline{S})$	R = 1011, S = 1101 → 1110
NOR	$R \leftarrow R \ NOR \ S$	$R = (R \ OR \ \overline{S})$	R = 1011, S = 1101 → 0000

4. Applications

- Used in ALU for performing logic functions.
- Used in data masking and bit testing.
- Essential in computer arithmetic and control.
- Used in digital signal processing and cryptography.

5. Advantages

- Simple and fast operations.
- Can be implemented using basic logic gates.
- Require less hardware compared to arithmetic.
- Useful for a wide range of digital systems.

⑥ Explain shifting microoperations including logical, circular, and arithmetic shifting operations.

Ans : Introduction

1. Shifting microoperations are used to move the bits of a register left or right by one or more positions.
   They are important for multiplication/division by 2, code conversion, and data alignment.
   Shifts are of three types : Logical, Circular and Arithmetic.

2. Logical Shift

(ii) Logical Left Shift (SHL)

- All bits are shifted left.
- 0 is shifted in from right end.
- Left most bit (MSB) is lost.
- Symbol : SHL R

Example : Let R = 1011011
After SHL by 1 → 0110110
After SHL by 2 → 1101100

(ii) Logical Right Shift (SHR)

- All bits are shifted right.
- 0 is shifted in from left end.
- Right most bit (LSB) is lost.
- Symbol : SHR R

Example : Let R = 1011011
After SHR by 1 → 0101101
After SHR by 2 → 0010110

> **Note :**
> - In logical shift, zeros are inserted in the vacant positions.
> - No wrap around of bits.

3. Circular (Rotate) Shift

(ii) Rotate Left (ROL)

- All bits are rotated left.
- Left most bit comes back at right end.
- No bit is lost.
- Symbol : SRL R

Example : Let R = 1011011
After ROL by 1 → 0110111
After ROL by 2 → 1101110

(ii) Rotate Right (ROR)

- All bits are rotated right.
- Right most bit comes back at left end.
- No bit is lost.
- Symbol : ROR R

Example : Let R = 1011011
After ROR by 1 → 1101011
After ROR by 2 → 1110110

> **Note :**
> - In circular shift, bits are rotated.
> - Useful in code conversion and crypto applications.

4. Arithmetic Shift

(ii) Arithmetic Left Shift (SAL)

- Similar to logical left shift.
- 0 is shifted in from right.
- Left most bit (MSB) is lost.
- Symbol : SAL R

Example : Let R = 1011011
After SAL by 1 → 0110110

(ii) Arithmetic Right Shift (SAR)

- For signed numbers (2's complement).
- Bits are shifted right.
- Sign bit (MSB) is replicated (remains same).
- Symbol : SAR R

Example : Let R = 1011011 (negative number)
After SAR by 1 → 1101101 (MSB=1 replicated)
After SAR by 2 → 1110110

> **Note :**
> - Arithmetic right shift preserves the sign of the number.

5. Summary Table

Type	Operation	Symbol	Direction	Bit shifted in	Bit lost	Example (R = 1011011)
Logical	Logical Left Shift	SHL	Left	0 (from right)	MSB	After SHL by 1 → 0110110
	Logical Right Shift	SHR	Right	0 (from left)	LSB	After SHR by 1 → 0101101
Circular (Rotate)	Rotate Left (ROL)	ROL	Left	MSB to LSB	None	After ROL by 1 → 0110111
	Rotate Right (ROR)	ROR	Right	LSB to MSB	None	After ROR by 1 → 1101101
Arithmetic	Arithmetic Left Shift (SAL)	SAL	Left	0 (from right)	MSB	After SAL by 1 → 0110110
	Arithmetic Right Shift (SAR)	SAR	Right	MSB replicated	LSB	After SAR by 1 → 1110101

6. Applications

- Multiply / Divide by 2 (left shift → ×2, right shift → ÷2).
- Fast arithmetic operations.
- Used in data alignment and code conversion.
- Important in CPU, DSP, and communication systems.

7. Advantages

- Fast operations (no arithmetic circuits required).
- Simple hardware implementation.
- Useful in many algorithms and data processing tasks.

(7) What is instruction code and explain the stored program concept.

Ans :

1. Instruction Code

- An instruction code is a binary pattern that specifies an operation to be performed by the computer.
- It consists of two parts :
  (i) Opcode (Operation Code) → specifies the operation.
  (ii) Address / Operand field → specifies the operand or the address of the operand.
- Example (8-bit instruction format) :

Opcode (4 bits)	Address / Operand (4 bits)

Example : 0101 1101

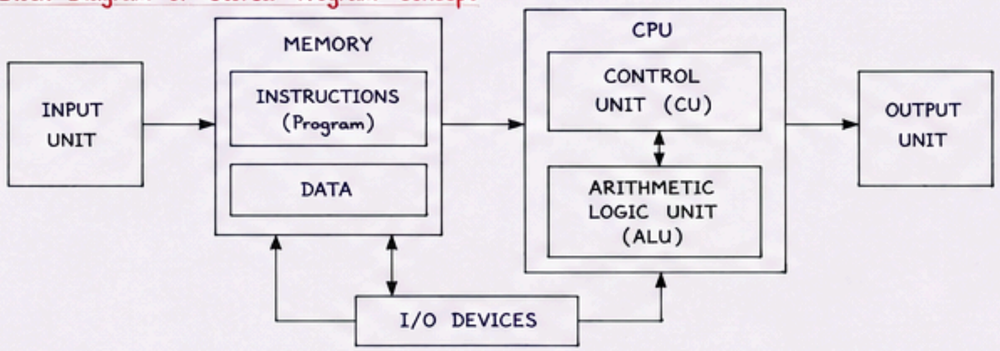
0101 → OPCODE, 1101 → ADDRESS

> **Key Points**
> - Instruction code tells the computer what to do and where to do it.
> - Opcode determines the type of operation.
> - Address / operand field locates the data or memory location.
> - Different instruction codes perform different operations.

2. Stored Program Concept

- The stored program concept was proposed by John von Neumann.
- In this concept, both instructions (program) and data are stored in the same read-write memory.
- The CPU fetches instructions from memory, decodes them, and executes the operation.
- After execution, the result is stored back in memory.
- Hence, the program can be modified like data.

3. Block Diagram of Stored Program Concept



4. Operation (How Stored Program Works)

(i) The program (instructions) and data are loaded into memory.
(ii) The Control Unit (CU) fetches an instruction from memory.
(iii) The instruction is decoded.
(iv) If the instruction requires data, it is read from memory.
(v) The ALU performs the specified operation.
(vi) The result is stored back in memory.
(vii) Next instruction is fetched. This process continues until HALT.

5. Advantages

- Easy to modify a program.
- Saves memory space.
- Enables complex operations.
- More flexible and efficient.
- Basis of modern computer design.

6. Summary

In the stored program concept, instructions are represented as binary codes and stored in the same memory along with data. The CPU fetches, decodes and executes the instructions sequentially. This concept is the foundation of all general-purpose computers.

(8) Describe the computer registers of a basic computer and state the function of each register.

Ans :

1. Introduction

A basic computer uses a set of registers to store data, addresses, instructions and control information temporarily. These registers are located in the CPU and are used to speed up operations.

2. Registers of a Basic Computer

Register	Abbreviation	Size (bits)	Function
Accumulator	AC	16	Holds one operand of arithmetic/logic operations and the result of the operation.
Memory Address Register	MAR	12	Holds the address of the memory location to be read from or written to.
Memory Buffer Register	MBR	16	Holds the data/instruction read from memory or data to be written into memory.
Program Counter	PC	12	Holds the address of the next instruction to be fetched from memory.
Instruction Register	IR	16	Holds the instruction just fetched from memory (contains opcode and address fields).
Temporary Register	TR	16	Holds temporary data during execution of instructions.
Input Register	INPR	8	Holds data received from an input device.
Output Register	OUTR	8	Holds data to be sent to an output device.
Sequence Counter	SC	4	Generates timing/control sequence for operations. (Counts timing steps T_0, T_1, ...)

3. Brief Description of Each Register

(i) AC : Used by ALU for arithmetic and logic operations.
(ii) MAR : Specifies the memory address for read/write operation.
(iii) MBR : Acts as a buffer between CPU and memory.
(iv) PC : Automatically incremented after each instruction fetch.
(v) IR : Decoded by Control Unit to determine the operation.
(vi) TR : Used to hold intermediate results.
(vii) INPR : Receives data from input device (e.g., keyboard).
(viii) OUTR : Sends data to output device (e.g., display).
(ix) SC : Synchronizes all micro-operations.

4. Summary of Functions

- AC, TR → Data manipulation
- MAR, MBR → Memory interface
- PC, IR → Instruction sequencing
- INPR, OUTR → I/O operations
- SC → Control and timing

5. Register Organization (Simplified View)

AC (16)	MAR (12)	MBR (16)	PC (12)	IR (16)	TR (16)	INPR (8)	OUTR (8)	SC (4)

Data Registers — Address / Control Registers — I/O & Control Registers

Summary : Registers are small, fast storage locations inside the CPU used to hold data, addresses, instructions and control information during execution. They play an important role in the speed and efficiency of the computer.

(9) Explain the basic computer instruction formats and classify different types of instructions.

Ans :

1. Basic Computer Instruction Formats

An instruction in a basic computer consists of an opcode that specifies the operation to be performed and an address field that specifies the location of the operand. The length of an instruction is fixed (16 bits in basic computer).

(ii) Memory Reference Instruction Format

15 – 12	11 – 0
Opcode (4 bits)	Address (12 bits)

(ii) Register Reference Instruction Format

15 – 12	11 – 0
1111 (4 bits)	Function (12 bits)

(iii) Input / Output Instruction Format

15 – 12	11 – 0
1110 (4 bits)	I/O Function (12 bits)

Key Points

- Opcode specifies the operation.
- Address field specifies the memory location of the operand.
- For memory reference instructions, 12-bit address is used.

Note :

In a basic computer system, the opcode 1111 is used for register reference instructions and 1110 for input/output instructions.

2. Classification of Instructions

Type of Instruction	Description	Examples
(i) Memory Reference Instructions	Operate on data in memory. Operand is in memory location specified by address field.	AND, ADD, LDA, STA, BUN, BSA, ISZ
(ii) Register Reference Instructions	Operate on data in CPU registers. No memory operand is specified.	CLA, CLE, CMA, CME, CIR, CIL, INC, SPA, SNA
(iii) Input / Output Instructions	Used for data transfer between CPU and I/O devices.	INP, OUT, SKI, SKO, ION, IOF

3. Memory Reference Instructions

- Involve an operand in memory.
- Format: Opcode (4 bits) + Address (12 bits)
- Examples:
  - LDA : Load accumulator
  - STA : Store accumulator
  - ADD : Add memory to AC
  - BUN : Branch unconditionally
  - BSA : Branch and save return address
  - ISZ : Increment memory and skip if zero

4. Register Reference Instructions

- Perform operations on CPU registers.
- Format: 1111 (opcode) + Function (12 bits)
- Examples:
  - CLA : Clear AC
  - CLE : Clear E
  - CMA : Complement AC
  - CIR : Circulate right
  - INC : Increment AC
  - SPA : Skip next instruction if AC positive

5. Input / Output Instructions

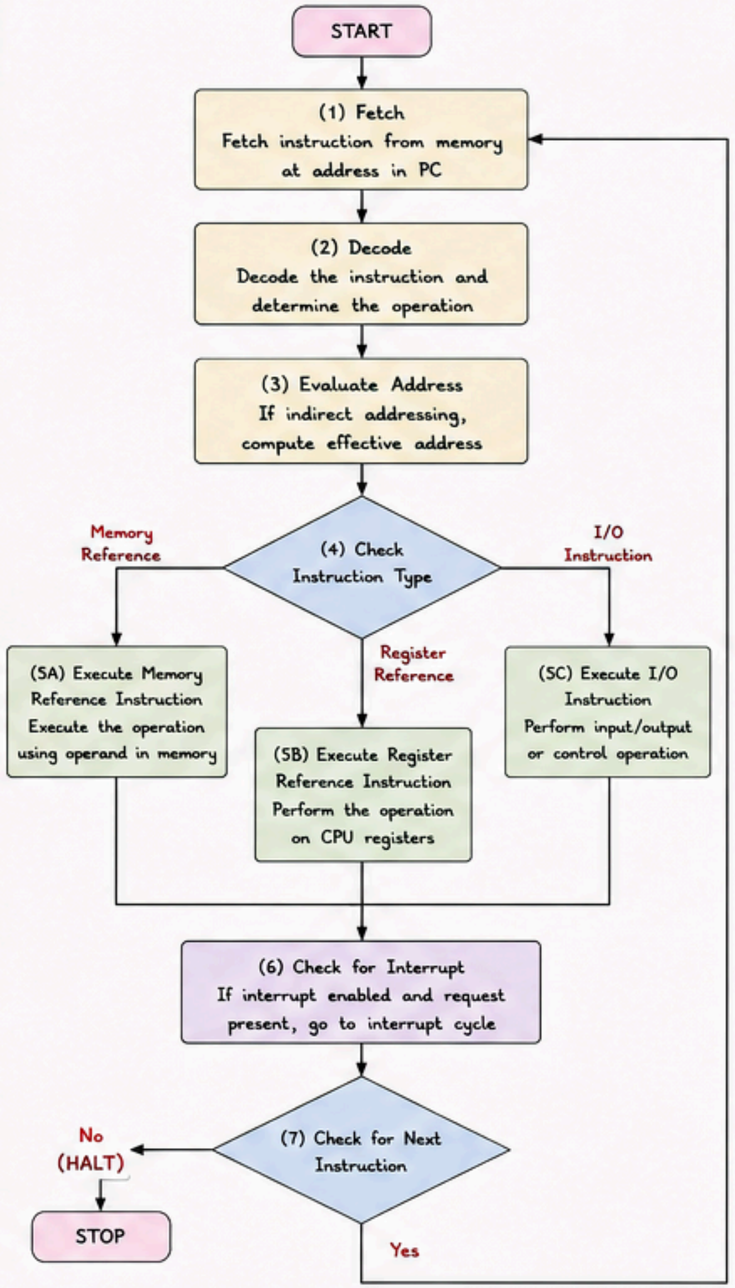
- Transfer data between CPU and I/O.
- Format: 1110 (opcode) + I/O function (12 bits)
- Examples:
  - INP : Input from device to AC
  - OUT : Output from AC to device
  - SKI : Skip if input flag = 1
  - SKO : Skip if output flag = 1
  - ION : Interrupt on
  - IOF : Interrupt off

Summary : Instruction format depends on the type of instruction. Instructions are classified as memory reference, register reference and I/O instructions. Each type has a specific format and performs unique operations.

(10) Draw the flowchart of instruction cycle and explain.

Ans :

1. Flowchart of Instruction Cycle



2. Explanation of Instruction Cycle

(1) Fetch

The instruction at the memory location pointed by the Program Counter (PC) is fetched and loaded into the Instruction Register (IR). Then PC is incremented.

(2) Decode

The opcode in IR is decoded to determine the type of instruction and required operation.

(3) Evaluate Address

If the instruction uses indirect addressing, the effective address is computed.

(4) Check Instruction Type

The decoded instruction is classified as Memory Reference, Register Reference, or I/O Instruction.

(5) Execute

- If Memory Reference: Operand is accessed from memory and operation is performed.
- If Register Reference: Operation is performed on CPU registers.
- If I/O Instruction: Data transfer with I/O device or control operation is performed.

(6) Check for Interrupt

If interrupts are enabled and an interrupt request is pending, the CPU branches to the interrupt cycle.

(7) Check for Next Instruction

If the program is not halted, the CPU returns to the fetch step for the next instruction. Otherwise, it stops.

Summary : The instruction cycle consists of fetch, decode, evaluate address, execute, check interrupt and repeat for the next instruction. It is the basic operation performed by the CPU for every instruction.

(11) Describe the interrupt cycle and explain how an interrupt is handled in the basic computer.

Ans :

1. Introduction

An interrupt is a request from an I/O device or a program to halt the current operation and execute a service routine (Interrupt Service Routine). After servicing the interrupt, the CPU resumes the interrupted program.

Key Points

- Interrupt improves CPU utilization.
- It is asynchronous with respect to the current instruction.
- The return address is saved in PC to resume the program.

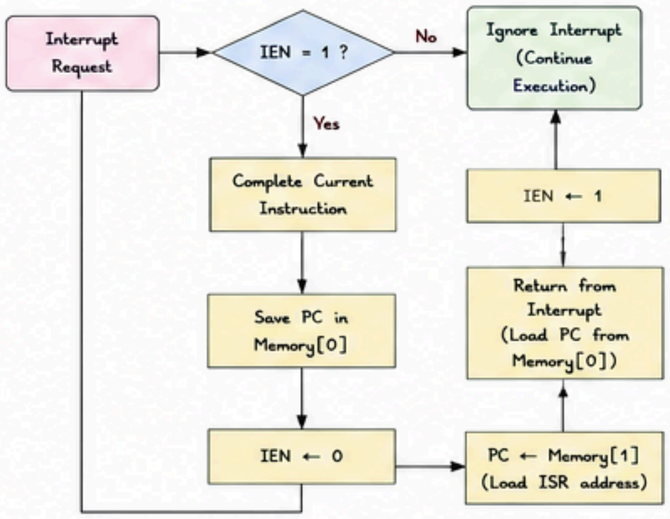
2. Interrupt Cycle (Steps)

(1) Interrupt Request : An I/O device or program sets the interrupt request line.
(2) Interrupt Enable Check : If interrupts are enabled (IEN = 1), CPU acknowledges.
(3) Finish Current Instruction : The CPU completes the execution of the current instruction.
(4) Save Return Address : The address of the next instruction (content of PC) is saved in memory location 0.
(5) Disable Interrupts : IEN is cleared (IEN = 0) to prevent nested interrupts.
(6) Load Interrupt Vector : The memory location 1 contains the address of the interrupt service routine. This address is loaded into PC.
(7) Execute ISR : The CPU transfers control to the ISR and executes it.
(8) Return from Interrupt : At the end of ISR, a return instruction is executed. The address in memory location 0 is loaded back into PC and IEN is set to 1.
(9) Resume Program : The CPU resumes execution of the interrupted program from the saved address.

Note :

- Memory[0] holds return address.
- Memory[1] holds interrupt service routine address.
- IEN = Interrupt Enable flip-flop.

3. Interrupt Cycle Flowchart



4. How Interrupt is Handled in Basic Computer

- The Basic Computer has a single interrupt input (INTR).
- An interrupt is accepted only if IEN = 1.
- The return address (PC) is stored in memory[0].
- The ISR starting address is stored in memory[1].
- Execution of ISR is same as a normal program.
- A return instruction (BUN I) is used at the end of ISR to return to the main program.

Summary : During an interrupt, the CPU saves the return address, disables interrupts, and jumps to the ISR whose address is in memory[1]. After executing the ISR, it restores the return address and resumes the main program.

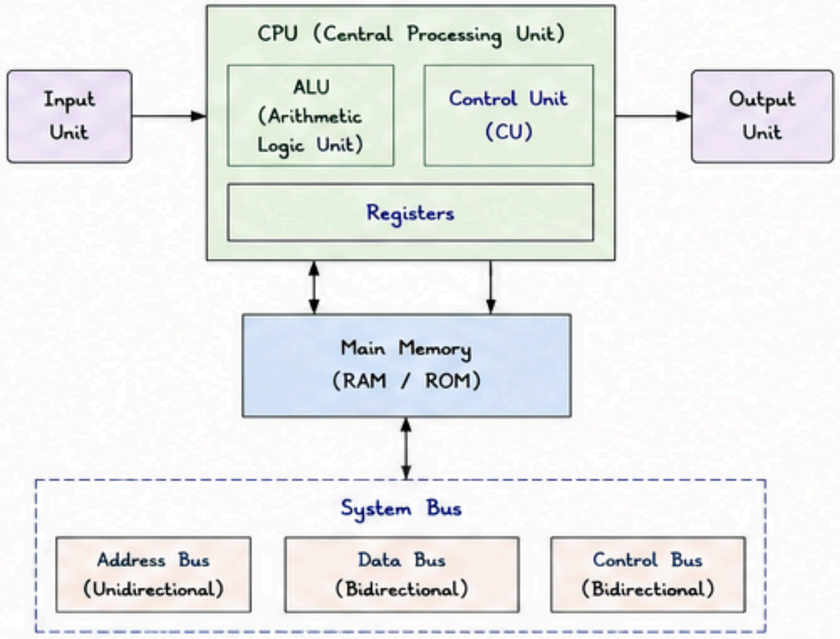
(12) Describe the basic architecture of the computer and its internal components.

Ans :

1. Introduction

A computer system can be divided into major functional units that work together to perform data processing tasks. The basic architecture is known as the von Neumann architecture.

2. Basic Architecture (Block Diagram)



Functions of Major Units

- Input Unit : Accepts data and instructions from external devices and converts them into binary form.
- Output Unit : Presents the results of processing to the user.
- Memory Unit : Stores programs, data, and intermediate results.
- CPU : Performs all data processing operations and controls the overall operation.
- System Bus : Provides communication among CPU, memory, and I/O devices.

3. Internal Components of CPU

Component	Description
ALU (Arithmetic Logic Unit)	Performs arithmetic operations (add, subtract, etc.) and logical operations (AND, OR, NOT, etc.).
Control Unit (CU)	Fetches instructions from memory, decodes them, and controls the sequence of operations in the computer.
Registers	Small, fast storage locations inside CPU used to hold data, addresses, and control information temporarily.
Program Counter (PC)	A special register that holds the address of the next instruction to be fetched.
Instruction Register (IR)	Holds the currently fetched instruction during execution.
Accumulator (AC)	A general purpose register used to store operands and results.
Memory Address Register (MAR)	Holds the address of the memory location to be accessed.
Memory Buffer Register (MBR)	Holds the data/instruction read from or written to memory.
I/O Registers	Hold data and control information for I/O operations.

4. Operation

- The CU fetches an instruction from memory using the address in PC.
- The instruction is loaded into IR and decoded.
- Required data is read from memory or input device.
- ALU performs the specified operation.
- The result is stored in a register or sent to output device.
- PC is updated to point to the next instruction.

Summary : The basic computer architecture consists of CPU, Memory Unit, Input Unit, Output Unit and System Bus. The CPU contains ALU, Control Unit and Registers which work together to execute instructions and process data.